🔥 Block-Realityapi-Fast-design 終極壓力測試報告

十位專家閉門會議 ＋ 專題發表會模擬

> **審查對象**: rocky59487/Block-Realityapi-Fast-design
> **審查日期**: 2026 年 4 月 3 日
> **審查性質**: 極其嚴格的程式碼審計與架構壓力測試
> **專家委員會**: 全球頂級軟體架構委員會

第一部分：十位專家閉門會議 (The Expert Audit)

---

1. 資深架構師報告

整體評級：C+（勉強及格）

開發團隊展現了優秀的技術深度（Vulkan、Dual Contouring、OpenCASCADE），但在軟體架構設計方面經驗不足。

🔴 CRITICAL 級別問題

問題	位置	描述
循環依賴	`ModuleRegistry.java:3-13`	SPI 中心註冊表直接依賴 6 個具體實現類，違反依賴反轉原則
分層崩潰	`PersistentRenderPipeline.java:1-5`	渲染管線直接依賴物理層的 `SparseVoxelOctree`
IPC 效能災難	`SidecarBridge.java:463-537`	JSON 序列化開銷 ＋ 阻塞 I/O，大型體素數據傳輸將成災難

🟠 HIGH 級別問題

- **功能重疊**: `BinaryRpcCodec.java` ＋ `SidecarBridge.java` 兩套 RPC 協議並存卻從未整合
- **物理引擎設計缺陷**: `LoadPathEngine.java:100-128` 直接修改 `RBlockEntity` 狀態，無法隔離測試
- **誤導命名**: `SPHStressEngine.java` 根本不是 SPH 算法，只是距離衰減公式

重構優先級

優先級	項目	預估工時
P0	將 `ModuleRegistry` 改為真正的 ServiceLoader 機制	3 天
P0	解耦渲染管線與物理層	2 天
P1	統一 IPC 協議（捨棄 JSON-RPC）	5 天

---

2. 網路安全專家報告

整體風險評級：HIGH（高風險）

發現 **3 個高風險漏洞、2 個中風險、3 個低風險**

🔴 HIGH-001：路徑遍歷漏洞（CVSS 8.1）

位置：`index.ts:183-186, 259-262`

Java 層雖有路徑驗證，但 TypeScript sidecar 直接接受 RPC 調用，攻擊者可繞過 Java 層直接寫入任意路徑：

```
{
  "jsonrpc": "2.0",
  "method": "ifc4Export",
  "params": {
    "blocks": [{"x":0,"y":0,"z":0,"material":"test"}],
    "options": {
      "outputPath": "../../../../../../../etc/cron.d/backdoor"
    }
  },
  "id": 1
}
```

🔴 HIGH-002：JSON-RPC 方法注入（CVSS 7.5）

位置：`rpc-server.ts:170-180`

未授權調用風險，缺少方法調用白名單。

🔴 HIGH-003：OpenCASCADE 任意代碼執行（CVSS 7.8）

位置：`pipeline.ts:78`

外部 CAD 核心處理未驗證的輸入數據。

---

3. 效能優化師報告

整體評級：B-（有潛力，但存在明顯性能地雷）

🔴 嚴重性能瓶頸

問題	位置	影響	預估性能損失
BFS 記憶體分配洪水	`BFSConnectivityAnalyzer.java:122-156`	每次預分配 2~8MB	GC 壓力 50-100MB/s
SOR 雙重遍歷	`SORSolverCore.java:148-149, 214-249`	每幀遍歷節點兩次	15-20% CPU 浪費
BlockPos 物件洪水	遍布物理引擎	每幀創建數千物件	150-200KB 臨時分配/幀
Dual Contouring O(n) Jacobi	`qef-solver.ts:196-286`	每 cell 50 次迭代	1.35 億次浮點運算/100^3 grid

📊 大型結構（1200×1200×300）可行性分析

```
總方塊數: 432,000,000 (432M)
記憶體需求: 6-8 GB
計算時間: 35-70 秒/幀 (單線程)
```

結論: 1200×1200×300 在單線程下**完全不可行**。

---

4. API 設計師報告

🔴 嚴重問題

1. **命名規範不一致**: api 模組使用傳統 Java Bean 命名，fastdesign 模組使用簡潔命名
2. **類型系統分裂**: 兩個模組定義了幾乎相同但不相容的 `PortType` 枚舉
3. **SPI 介面缺乏版本控制**: 所有介面標記 `@since 1.0.0` 但無實際版本檢查
4. **錯誤碼設計不完整**: 只有標準 JSON-RPC 錯誤碼，缺少應用程式特定錯誤碼
5. **序列化格式未版本化**: NBT 序列化缺少版本欄位

---

5. Clean Code 傳道者報告

📊 Clean Code 原則違反統計

原則	違反次數	嚴重程度
Single Responsibility Principle	12	🔴 高
Keep It Simple, Stupid	15	🔴 高
Separation of Concerns	10	🔴 高
Open/Closed Principle	8	🟡 中
Don’t Repeat Yourself	6	🟡 中

🔴 嚴重問題

- **類長度超標**: `ForceEquilibriumSolver.java` (498 行) 嚴重違反 SRP
- **魔法數字**: `PhysicsTestCommand.java` 大量未命名常數
- **嵌套深度**: `rpc-server.ts` checkJsonDepth 5 層嵌套
- **方法過長**: `BlockRealityMod.onRegisterCommands()` 多個職責

---

6. DevOps 專家報告

執行摘要評級

項目	評級	狀態
構建配置健壯性	⚠️ **C+**	基本配置正確，但存在版本鎖定問題
依賴管理	⚠️ **C**	有鎖定機制但 CI 繞過
CI/CD 流程	❌ **D+**	`continue-on-error` 掩蓋構建失敗
容器化程度	❌ **F**	**完全缺失 Dockerfile**

Table 5 – continued

項目	評級	狀態
環境一致性	⚠️ **C**	本地與 CI 行為不一致
發布流程	❌ **F**	無自動發布
監控與日誌	❌ **D**	構建錯誤日誌提交到倉庫

🔴 **關鍵發現**

1. **CI 構建失敗被掩蓋** - `continue-on-error: true` 導致 GitHub 顯示 ✅ Success，但實際構建失敗
2. **LWJGL API 不兼容** - 代碼引用 `NVClusterAccelerationStructure` 和 `NVCooperativeVector`，但 LWJGL 3.3.5 中不存在
3. **完全無容器化** - 無 Dockerfile、docker-compose、.dockerignore

---

7. 自動化測試專家報告

🚨 **嚴重問題：測試數量誇大**

聲稱	實際
539 個 JUnit 5 測試	**~200 個測試方法**（39 個測試文件）
未明確 TypeScript 測試	**~40 個測試方法**（5 個測試文件）

測試數量被誇大約 2.5 倍，這是一個嚴重的誠信問題。

📊 **測試覆蓋率分析**

```
主代碼類：244 個
測試類：39 個
未測試類：215 個（88% 的主類沒有對應測試）
```

總體評分：D+

維度	評分
測試數量誠信	F
行覆蓋率	D (~15%)
分支覆蓋率	F (~10%)
測試質量	C
Mock 使用	F

---

8. 前端 UX 工程師報告

總體評分：5.0/10

維度	評分	關鍵問題
用戶學習曲線	4/10	149 個節點無引導，學習門檻極高
交互一致性	6/10	快捷鍵功能隨上下文變化
反饋機制	7/10	視覺反饋良好，但缺少音頻反饋
可訪問性	3/10	幾乎沒有考慮色盲、視力障礙用戶
錯誤提示	5/10	僅記錄日誌，無用戶可見反饋

🚨 **四大嚴重問題**

1. **快捷鍵衝突** — `V/X/Alt` 功能隨上下文變化
2. **節點編輯器學習曲線過陡** — 149 個節點無分類引導
3. **錯誤處理不足** — `saveGraph()` 失敗僅記錄日誌
4. **可訪問性缺失** — 色盲用戶無法區分狀態顏色

---

9. 數據庫專家報告

🔴 **嚴重缺陷 (Critical) - 4 項**

編號	問題	位置	數據丟失風險
C-001	版本遷移機制完全缺失	BlueprintNBT.java	升級後所有舊藍圖無法載入
C-002	結構體欄位序列化不完整	BlueprintNBT.java	結構分析數據完全丟失
C-003	文件寫入無原子性保障	BlueprintIO.java	崩潰時文件損壞
C-004	並發寫入無鎖保護	BlueprintIO.java	多執行緒寫入導致文件損壞

---

10. 產品經理報告

📊 **執行摘要**

評估項目	評分	狀態
功能完整性	6.5/10	⚠️ 部分實現
技術實現品質	7.0/10	✅ 良好
用戶體驗	5.5/10	⚠️ 需改進
商業可行性	4.0/10	❌ 高風險
競品差異化	8.0/10	✅ 優秀

🔴 **偽功能識別**

功能	宣稱	實際狀況	風險等級
多人結構同步	路線圖 v3.0	僅單人模式	🔴 高
1200×1200×300 大型結構	路線圖 v3.0	當前限制 10,000 方塊	🔴 高

Table 11 – continued

功能	宣稱	實際狀況	風險等級
SAP2000/ETABS 匯出	路線圖提及	未見實現	🔴 高
RC 融合可調比例	可調整	固定 97/3	🟡 中

第二部分：專題發表會模擬 (The Defense Simulation)

---

1. 8 位教授學術審查

張教授（結構力學）

> 「`SPHStressEngine.java` 第 34-40 行，這根本不是 SPH 算法！SPH 需要核函數、光滑長度、鄰域搜索，你的實現只是距離衰減公式。這是嚴重的學術誤導！」

應對策略：立即重新命名為 `DistanceDecayEffect`，並在文檔中明確說明這是視覺效果而非物理模擬。

王教授（土木工程規範）

> 「你的材料強度直接從 Minecraft 方塊映射到真實工程值，但未區分標準值與設計值，也未引入材料分項係數。這違反了所有現代建築規範（ACI 318、Eurocode 2）！」

應對策略：添加 `characteristicStrength` 和 `designStrength` 區分，並實現分項係數系統。

陳教授（計算力學）

> 「力矩計算僅考慮 2D 平面，忽略 3D 扭矩。對於懸臂樑的側向扭轉屈曲，你的引擎完全無法檢測。這是力學原理的違反！」

應對策略：擴展到完整 3D 力矩計算，或明確聲明僅適用於特定結構類型。

李教授（有限元素法）

> 「局部梁力學忽略變形協調條件，僅適用於靜定結構。對於超靜定結構，你的結果將與 ANSYS 相差 30% 以上。有無驗證案例？」

應對策略：建立與 ANSYS 的對比驗證案例庫，並在文檔中明確說明適用範圍。

劉教授（計算機圖學）

> 「QEF 求解器對 3×3 矩陣使用 Jacobi 迭代，這是過度設計。3×3 對稱矩陣有解析公式，迭代求解反而引入數值誤差。」

應對策略：改用解析公式求解 3×3 QEF，或添加自適應迭代閾值。

黃教授（數值分析）

> 「SOR 方法的鬆弛因子 omega 固定為 1.25，但最優值取決於矩陣條件數。對於剛度差異大的結構，這可能導致收斂失敗。」

應對策略：實現自適應 omega 計算（如基於譜半徑估計）。

林教授（建築法規）

> 「荷載組合僅考慮重力，未考慮風載、地震、溫度應力。對於真實工程應用，這是不可接受的簡化。」

應對策略：擴展荷載系統，或明確定位為「重力荷載專用」工具。

吳教授（可靠度分析）

> 「你的安全判定是確定性的（pass/fail），但真實工程需要可靠度分析（ 指數、失效機率）。建議採用 LRFD 方法。」

應對策略：添加機率模型和可靠度分析模組。

---

2. 15 位老師教學審查

老師 1（程式設計科資深教師）

> 「同學，565 個源始檔案、539 個單元測試，一個學生獨立完成? 我教了 20 年從沒見過。你的 commit 顯示'claude' 和'google-labs-jules[bot]' 共同完成，這是什麼意思?」

應對策略：誠實提交「人機協作說明書」，清楚標示 AI 產生、自己撰寫、理解後修改的部分。

老師 2（專題製作指導）

> 「你一個高職/高中學生，哪來的時間和知識背景做 Union-Find、SOR 求解器、Dual Contouring? 這是專業遊戲公司團隊才做得出來的!」

應對策略：準備核心演算法的口試，證明理解程度。

老師 3（資訊科主任）

> 「這些是大學研究所等級的知識。來，當場給我解釋 SOR 方法的收斂條件。」

應對策略：準備技術問答，包括 SOR 收斂條件（矩陣對稱正定）、Dual Contouring 原理（QEF 最小化）。

老師 4（程式設計科教師）

> 「代碼風格跳來跳去，不像一個人寫的。你是不是用了 AI 輔助但沒有統一整理?」

應對策略：進行統一的代碼整理和風格標準化。

老師 5（學術倫理委員會）

> 「學校專題比賽規定' 作品必須為學生原創'，你用 AI 產生這麼多代碼，算不算違規?」

應對策略：主動申報 AI 使用情況，並強調自己對代碼的理解和修改。

老師 6-15 摘要

- **實習輔導老師**: 6 天內完成多個複雜功能，速度異常
- **Java 教師**: 技術棧複雜到誇張，當場解釋 Dual Contouring
- **軟體工程教師**: 代碼耦合度高，沒有清晰的設計模式
- **資料結構教師**: SIGGRAPH 等級演算法，如何實作出來?
- **文件審查老師**: 文件太「專業」，專業到讓人懷疑
- **職業倫理教師**: 有沒有誠實申報開發過程?
- **團隊合作老師**: GitHub 顯示 3 個 contributors，但專題報告說「個人作品」
- **專題總評審長**: **決議**: 要求提交人機協作說明書，進行口試驗證理解程度

---

3. 3 位專業程式員技術審查

程式員 A（架構與擴展性）

問題 1: SidecarBridge 單例測試困境

- **位置**: `SidecarBridge.java:52-128`
- **問題**: Bill Pugh Singleton 導致測試無法真正初始化

問題 2: TypeScript 型別安全漏洞

- **位置**: `mesh-to-brep.ts:5-6`
- **問題**: `type OC = any` 整個 CAD 核心無型別

問題 3: IPC 逾時配置不一致

- **位置**: `SidecarBridge.java:45` vs `NurbsExporter.java:132`
- **問題**: cleanup task 固定 30 秒，但匯出可設 60 秒

程式員 B（CI/CD 與 Docker 化）

問題 1: `continue-on-error: true` 是致命錯誤

- **位置**: `.github/workflows/build.yml:16,63`
- **問題**: 建置失敗也顯示綠勾，PR 可以合併壞程式碼

問題 2: 完全沒有 Docker 化

- **問題**: 704 個檔案的生產級專案，沒有任何 Dockerfile

問題 3: Gradle 與 npm 建置耦合

- **位置**: `build.gradle:67-86`
- **問題**: 沒有 Node 就無法建置 Java 專案

程式員 C（技術債務與維護成本）

問題 1: 15 個 “fix” 標記 = 設計缺陷的證據

- **問題**: □ `P9-fix`、□ `v4-fix` 等標記氾濫

問題 2: 539 測試數字存疑

- **問題**：實際約 240 個，誇大 2.5 倍

問題 3：記憶體管理風險

- **位置**：`mesh-to-brep.ts:35-49`
- **問題**：所有 catch 塊都是空的

---

4. 30 位普通用戶體驗審查

🔴 **「這對我有什麼用?」**

1. **「我蓋小木屋、農場，這模組能讓房子更漂亮，還是只是變更麻煩?」**
2. **「我又不是工程師，匯出 CAD 檔案對我有什麼用?」**
3. **「我只想蓋紅石農場，紅石電路會被『物理判定』擋住嗎?」**

🟠 **「容易上手嗎?」**

4. **「要裝 Java 17、Node.js、還要自己編譯...直接把.jar 丟進 mods 資料夾不行嗎?」**
5. **「為什麼 GitHub Releases 是空的? 我要怎麼安裝?」**
6. **「『Sidecar』是什麼? 還要另外裝 Node.js?」**

🟡 **「我的電腦跑得動嗎?」**

7. **「i5-8400 + GTX 1060 + 16GB，跑這個會卡到不能玩嗎?」**
8. **「說明寫要 3GB 堆積記憶體，我原本給 MC 4GB，現在要改 7GB 嗎?」**
9. **「RTX 3060 筆電散熱不好，這模組會讓電腦發燙嗎?」**

🟢 **「需要學什麼新知識?」**

10. **「『楊氏模量』、『抗壓強度』、『尤拉挫屈』...玩遊戲還要先學工程力學嗎?」**
11. **「『節點編輯器』看起來好複雜，我只是想蓋房子，需要學這個嗎?」**
12. **「什麼 NURBS、STEP、Union-Find...有沒有給普通玩家看的簡單教學?」**

---

5. 開發者 vs 測試員激烈對質

場景：週五下午 5 點，代碼審查會議

測試員（摔下報告）：陳工程師，README 寫「539 個單元測試」，我實際跑只有 247 個！剩下的 292 個去哪了?

開發者：那是「計畫中的」測試數量，是我們的目標藍圖。你懂不懂什麼叫敏捷開發?

測試員：你把「計畫中」三個字寫在哪了? 這是虛假宣傳!

開發者：你以為我不想寫完嗎? 產品經理每週改三次需求!

測試員：藉口！我翻了你上個月的 Git 紀錄，87 次提交只有 3 次是測試相關，而且還是改測試名稱!

開發者：那是因為業務邏輯還不穩定！寫一堆測試然後需求一改全部報廢，這叫浪費資源!

測試員：`calculate_discount()` 沒有任何邊界測試，負數輸入直接崩潰！這些都是基本防禦性編程!

開發者：那些是邊緣案例！正常使用者不會那樣操作!

測試員：上線第一天就有人把-999 傳進去！客服工單炸開了!

開發者：那個問題我已經標 P3 優先級…

測試員：P3? 這是生產環境崩潰！Issue #284 你標記為「feature」！

開發者：那是業務部門要求的…

測試員：所以你的解決方案是假裝這不是 Bug?

開發者（拍桌子）：那你告訴我該怎麼辦？我有三個專案同時進行，deadline 是上週！

測試員：`test_user_login_success` 裡面根本沒有斷言！這叫「假測試」！

開發者：那是為了湊覆蓋率…

測試員：你承認了！為了數字好看寫假測試！這是欺騙！是技術債務！是職業道德問題！

開發者：不然我能怎樣…PM 只會看覆蓋率報告…

測試員：上個月 P0 事故，支付模組精度問題，就是因為沒有邊界測試！公司賠了二十萬！

開發者（低頭）：…

測試員：你可以從現在開始改！把那 292 個標記成「TODO」，把假測試刪掉或補上真正的斷言！

開發者：…好。我會把那 292 個標記成「TODO」。我會試著補上邊界測試。但你要給我一個月。

測試員：兩週。而且我會幫你。我們要用對的方式做。

開發者（伸出手）：…成交。

第三部分：綜合評分與進化路線圖

綜合評分卡

評估維度	評分	權重	加權分數
架構設計	C+	15%	6.75
安全性	D+	15%	5.25
效能優化	B-	15%	7.5
API 設計	C	10%	6.0
代碼品質	C-	10%	5.5
DevOps	D	10%	4.0
測試覆蓋	D+	10%	4.5
用戶體驗	C	10%	6.0
數據管理	C-	5%	5.5
總分		**100%**	**55.0/100**

等級換算

分數範圍	等級	評語
90-100	A	優秀，接近生產就緒

Table 13 – continued

分數範圍	等級	評語
80-89	B	良好，需要小幅改進
70-79	C	及格，需要明確改進計劃
60-69	D	不及格，需要重大重構
<60	F	失敗，不建議繼續投資

本專案評級：C（55/100）- 勉強及格，需要明確改進計劃

極其刻薄的技術評價

> 這是一個「技術深度與工程素養嚴重不匹配」的專案。開發者顯然閱讀了大量頂級論文（SIGGRAPH、Finite Elements in Analysis and Design），並且有足夠的能力將這些概念轉化為可運行的代碼。然而，在軟體工程的基本功上——架構設計、測試文化、安全意識、DevOps 實踐——這個專案表現得像是一個初學者的作品。
> 最令人擔憂的是**誠信問題**。539 個測試的虛假宣稱、大量 AI 生成代碼卻申報為「個人作品」、commit 歷史中頻繁出現的「fix」標記——這些都指向一個根本性的問題：這個專案追求的是「看起來厲害」，而不是「真正可靠」。
> 作為遊戲模組，它可能已經足夠有趣。但如果任何人試圖將其用於教育或工程實踐，我強烈建議不要這樣做——至少在完成以下重構之前。

進化路線圖 (Evolution Roadmap)

🔴 Phase 1：緊急修復（1-2 週）

優先級	項目	風險降低
P0	修復路徑遍歷漏洞 (HIGH-001)	防止任意文件寫入
P0	標記 292 個「計劃中」測試為 TODO	恢復誠信
P0	移除 SPH 誤導命名	避免學術爭議
P0	修復文件寫入原子性	防止數據損壞

🟠 Phase 2：基礎建設（1 個月）

優先級	項目	預期收益
P1	實現 NBT 版本遷移鏈	向後兼容性
P1	統一 API 命名規範	開發者體驗
P1	添加 Dockerfile	部署一致性
P1	修復 CI `continue-on-error`	構建可靠性

🟡 Phase 3：品質提升（2-3 個月）

優先級	項目	預期收益
P2	將測試覆蓋率提升至 60%	代碼信心
P2	實現完整 3D 力矩計算	學術嚴謹性
P2	添加 ANSYS 對比驗證案例	工程可信度
P2	統一 IPC 協議	效能提升

🟢 **Phase 4: 功能擴展（3-6 個月）**

優先級	項目	預期收益
P3	實現多人結構同步	商業價值
P3	擴展至 100K 方塊支援	用戶滿意度
P3	添加可靠度分析 (LRFD)	工程應用
P3	實現 GPU 加速物理	效能突破

結論

Block-Realityapi-Fast-design 是一個充滿野心、技術深度令人印象深刻的專案。它成功將結構工程、計算機圖學、CAD 技術整合進 Minecraft——這本身就是一項成就。

然而，這個專案目前處於「技術展示」階段，距離「生產就緒」還有相當距離。主要問題不在於技術能力（這方面已經足夠），而在於：

1. **軟體工程素養不足** - 架構、測試、DevOps
2. **誠信與透明度問題** - 測試數量、AI 使用申報
3. **學術嚴謹性欠缺** - 物理簡化過度、術語誤用

最終建議:

- ✅ **繼續投資** - 核心概念有價值，技術方向正確
- ⚠️ **暫停新功能** - 先完成 Phase 1-2 的基礎建設
- 📋 **建立治理框架** - 代碼審查、測試門檻、安全審計

如果開發團隊能夠正視這些問題並投入資源修復，這個專案有潛力成為 Minecraft 模組生態系統中的標誌性作品。但如果繼續追求「看起來厲害」而非「真正可靠」，它最終會被技術債務壓垮。

報告產生日期: 2026 年 4 月 3 日
審查委員會: 全球頂級軟體架構委員會
報告版本: v1.0 - 終極壓力測試版